Seven Recommenders on MovieLens-1M: Rating Prediction, Top-10 Ranking, and a Fairness Audit

CS 550 Course Project, Option 1: Recommender System

Daize Dong
dd1376
Rutgers University
New Brunswick, USA

Jiawei Wu
jw2070
Rutgers University
New Brunswick, USA

Abstract

We implement and compare seven recommender algorithms on the MovieLens-1M dataset, covering the full range from trivial baselines to graph neural networks: GlobalMean, BiasOnly, ItemKNN, UserKNN, biased Matrix Factorization (MF), Neural Matrix Factorization (NeuMF), Bayesian Personalized Ranking Matrix Factorization (BPR-MF), and LightGCN. All seven share a single per-user 80/20 split and a single evaluation protocol, so differences come only from modelling choices. We report the two rating-prediction metrics required by the course specification (MAE, RMSE) and the four Top-10 ranking metrics (Precision@10, Recall@10, F_1@10, NDCG@10). As an optional trustworthiness task, we additionally audit every model for exposure fairness via Coverage, Gini, and long-tail share. Every model is written from NumPy and PyTorch primitives; no recommender-system library (Surprise, RecBole, LightFM, implicit, etc.) is imported. We find that the training loss matters far more than the architecture on this dataset (BPR-MF beats MF by 2.55× on P@10 with the same scoring function), that LightGCN does not beat BPR-MF at matched gradient budget, and that the two best ranking models are also the two worst on fairness.

1 Overview

This project implements seven recommender models on the MovieLens-1M dataset and compares them on two tasks required by the course specification: (i) rating prediction, evaluated with MAE and RMSE, and (ii) Top-10 item recommendation, evaluated with Precision@10, Recall@10, F_1@10, and NDCG@10. The seven models cover four families: simple baselines (GlobalMean, BiasOnly), memory-based collaborative filtering (UserKNN, ItemKNN), matrix factorization (MF, BPR-MF), a neural method (NeuMF), and a graph-based method (LightGCN). Every model is implemented from scratch using NumPy and PyTorch primitives; no recommender-system library is imported. We use PyTorch layers and optimizers (Embedding, Linear, Adam) as low-level building blocks only, per the course policy that forbids using packaged recommender algorithms directly.

2 Dataset and Preprocessing

We use the MovieLens-1M dataset, the "small" movie recommendation dataset listed in the project description. It contains 1,000,209 integer ratings on a 1 to 5 scale from 6,040 users on 3,706 movies, with each user rating at least 20 movies (the 20-core condition).

Preprocessing. We drop items with fewer than 5 ratings to stabilize similarity estimation, leaving N_u=6040 users and N_i=3416 items. User and item IDs are remapped to contiguous integer ranges to allow dense embedding tables.

Split. Following the course spec, for each user we uniformly sample 20% of their ratings as test and the remaining 80% as train, with random seed 42. Because every user has ≥ 20 ratings in the raw data and we do not drop users, every user has at least one training and at least one test rating, so no special casing is needed. The final split is $|\mathcal{D}_{\text{train}}|$=799,721 and $|\mathcal{D}_{\text{test}}|$=199,890, with 95.16% sparsity. Review text is not used; timestamps are not used.

3 Methods

Notation: user u, item i, observed rating r_{ui}, global mean μ, user / item biases b_u, b_i, user and item latent factors $p_u, q_i \in \mathbb{R}^d$.

3.1 Baselines

GlobalMean predicts $\hat{r}_{ui} = \mu$ for all pairs. **BiasOnly** predicts $\hat{r}_{ui} = \mu + b_u + b_i$, where the biases are learned by 15 alternating closed-form ridge updates with $\lambda = 5$.

3.2 Neighbourhood CF

We use shrinkage-adjusted cosine similarity on mean-centred ratings. For the item side:

$$s_{ij} = \frac{\sum_{u \in \mathcal{U}_{ij}} \tilde{r}_{ui}\tilde{r}_{uj}}{\|\tilde{r}_{\cdot i}\|\|\tilde{r}_{\cdot j}\|} \cdot \frac{|\mathcal{U}_{ij}|}{|\mathcal{U}_{ij}| + \alpha}, \qquad \tilde{r}_{ui} = r_{ui} - \bar{r}_u,\ \alpha = 100. \quad (1)$$

We keep the top $k = 50$ neighbours per item. The prediction is:

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{j \in N_k(i) \cap \mathcal{I}_u} s_{ij}\tilde{r}_{uj}}{\sum_{j \in N_k(i) \cap \mathcal{I}_u} |s_{ij}|}. \quad (2)$$

UserKNN works the same way but on the user side.

3.3 Biased Matrix Factorization (MF)

The prediction is $\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$, trained with L2-regularized MSE loss. We use $d = 64$, 20 epochs of Adam with learning rate 10^{-2}, batch size 8192, and $\lambda = 10^{-4}$.

3.4 Neural Collaborative Filtering (NeuMF)

NeuMF has two parallel branches: a GMF branch that computes $p_u^g \odot q_i^g$, and an MLP branch on the concatenation $[p_u^m \| q_i^m]$ with hidden sizes (64, 32, 16), each layer followed by ReLU and Dropout(0.1). The outputs of the two branches are concatenated and passed through a linear layer. The original NeuMF uses binary cross-entropy for implicit feedback; we use MSE instead because the task here is explicit rating prediction.

3.5 BPR-MF

BPR-MF trains on triples (u, i, j) where i is an observed item and j is an unobserved item sampled uniformly with rejection against the positive set. The loss is:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i,j)} \log \sigma(\hat{x}_{ui} - \hat{x}_{uj}) + \lambda \|\Theta\|^2, \qquad \hat{x}_{ui} = p_u^\top q_i + b_i. \quad (3)$$

We use $d = 64$, 20 epochs, batch size 4096, Adam with learning rate 10^{-2}, and $\lambda = 10^{-5}$.

3.6 LightGCN

LightGCN removes feature transforms and non-linearities from the standard GCN, keeping only weighted-average propagation on the user-item bipartite graph. Let $\mathbf{A}$ be the adjacency matrix and $\hat{\mathbf{A}} = \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ be its symmetric-normalized form. Each layer computes $\mathbf{E}^{(\ell)} = \hat{\mathbf{A}}\mathbf{E}^{(\ell-1)}$, and the final embedding averages over all layers:

$$\mathbf{E}^* = \frac{1}{L+1} \sum_{\ell=0}^{L} \mathbf{E}^{(\ell)}. \quad (4)$$

Training uses the same BPR loss. We set $L = 3$ and $d = 64$. Because the first LightGCN run used a much larger batch size than BPR-MF and therefore many fewer gradient steps, we run LightGCN twice: once at 15 epochs with batch size 16,384, and once at 20 epochs with batch size 4,096, which matches the gradient-step count of BPR-MF. The matched run is the one cited in the main results table; the short run is reported in Table 2 for comparison.

4 Evaluation Protocol

Rating prediction. MAE $= \frac{1}{|\mathcal{D}|} \sum |r - \hat{r}|$ and RMSE $= \sqrt{\frac{1}{|\mathcal{D}|} \sum (r - \hat{r})^2}$, with predictions clipped to $[1, 5]$.

Top-N recommendation. For each user we score all items, mask out items already in the user's training set (so we never recommend an item the user has already rated), take the top $K = 10$, and compute Precision@K, Recall@K, F_1@K, and NDCG@K against the user's held-out test items. Metrics are averaged over all users who have at least one test item.

Why two models are N/A on MAE/RMSE.. BPR-MF and LightGCN are trained with the BPR pairwise loss and produce arbitrary real-valued scores rather than numbers on the 1 to 5 rating scale. Clipping those scores to $[1, 5]$ would produce numerically defined but semantically meaningless MAE/RMSE values that only reflect the scale mismatch, not prediction quality. We therefore mark those cells as N/A in the rating prediction table and evaluate BPR-MF and LightGCN only on the Top-10 task, which is the task those losses were designed for. The six remaining models are evaluated on both tasks.

5 Results

5.1 BPR-MF vs. MF: Loss Function Matters

BPR-MF and MF use the *same* scoring function $p_u^\top q_i + b_i$ but different loss functions. BPR-MF beats MF by 2.55× on P@10 (0.286 vs. 0.112) and 2.57× on NDCG@10 (0.320 vs. 0.125). The reason is that MSE training tries to fit every rating value (including 2s and 3s), which is not directly useful for ranking. BPR training focuses on the relative order between observed and unobserved items, which is exactly what the ranking metrics measure.

5.2 kNN: Good on Rating, Bad on Ranking

UserKNN and ItemKNN have competitive MAE (0.707 and 0.714 vs. MF's 0.704), but they perform poorly on ranking. For items a user has not rated, the kNN prediction tends to fall back toward the user's mean rating: the neighbours that drive the prediction are mostly items the user has already rated, and those are masked out in Top-N evaluation. The remaining scores do not vary enough to produce a good ranking.

5.3 LightGCN vs. BPR-MF

LightGCN performs worse than BPR-MF on all ranking metrics (P@10: 0.183 vs. 0.286; NDCG@10: 0.204 vs. 0.320). The first run used 15 epochs at batch size 16,384 (about 720 gradient steps), which is far fewer than BPR-MF's roughly 3,900 steps. To rule out the possibility that LightGCN was simply undertrained, we re-ran with matched gradient steps (20 epochs, batch size 4,096). The numbers do not change (Table 2).

The gap is not caused by insufficient training. The likely reasons are:

(1) We used $L = 3$ layers, but $L = 2$ is the optimum reported in the original paper for ML-1M. We did not sweep this hyperparameter.
(2) Although the overall sparsity is 95.16%, every user has at least 20 ratings, so a plain factor model already has enough direct signal per user. The extra neighbour-of-neighbour information from graph convolution may not add much in this setting.
(3) Our regularization ($\lambda = 10^{-4}$, Adam defaults) was not tuned.

5.4 NeuMF

NeuMF is slightly worse than MF on ranking (NDCG@10: 0.098 vs. 0.125) and about the same on rating prediction (MAE: 0.704 vs. 0.704). The extra neural layers do not improve results on this dataset.

6 Optional Task: Exposure Fairness Audit

This section addresses the optional trustworthiness task (b), *Fairness and Unbiases of Recommender Systems*, as listed in the project description. Beyond ranking accuracy, we check whether each model concentrates its recommendations on the most popular items. For each model at $K = 10$, we measure:

- **Coverage**: the fraction of items in the catalogue that appear in at least one user's Top-10 list.
- **Gini**: the inequality of per-item exposure counts. 0 means every item is exposed equally, 1 means all exposure goes to one item.
- **Long-tail Share**: the fraction of recommendation slots that go to items outside the top 20% most popular items in the training set.

LightGCN places every Top-10 slot on popular items (Long-tail Share = 0.000) and touches only 2.2% of the catalogue. BPR-MF is similar (Long-tail Share = 0.005, Coverage = 20.4%). The two best-performing models on ranking accuracy are the two worst on fairness. On the other hand, UserKNN devotes 87% of its list to tail

Table 1: Test-set results on MovieLens-1M. Lower is better for MAE/RMSE; higher is better for P/R/F_1/NDCG. BPR-MF and LightGCN do not predict explicit ratings and are only evaluated on Top-N.

Model	MAE	RMSE	P@10	R@10	F_1@10	NDCG@10	t(s)
GlobalMean	0.9331	1.1161	0.0089	0.0022	0.0035	0.0088	0.0000
BiasOnly	0.7156	0.9050	0.0525	0.0196	0.0285	0.0492	0.4000
ItemKNN	0.7142	0.9308	0.0132	0.0064	0.0086	0.0140	1.8200
UserKNN	0.7069	0.9182	0.0124	0.0044	0.0065	0.0129	3.9700
MF	0.7036	0.8873	0.1120	0.0383	0.0571	0.1247	33.5000
NeuMF	0.7037	0.9043	0.0904	0.0311	0.0463	0.0977	87.5000
BPR-MF	N/A	N/A	0.2861	0.1065	0.1552	0.3202	48.9900
LightGCN	N/A	N/A	0.1824	0.0671	0.0981	0.2041	2270.8200

Table 2: LightGCN under two training budgets on ML-1M. Gradient steps ≈ epochs × $|\mathcal{D}_{\text{train}}|$/batch. The matched run uses the same batch size and epoch count as our BPR-MF run.

Config	Batch	Steps	P@10	NDCG@10
LightGCN, short	16 384	∼720	0.1826	0.2041
LightGCN, matched	4 096	∼3 900	0.1824	0.2041
BPR-MF (ref.)	4 096	∼3 900	**0.2861**	**0.3202**

Table 3: Exposure-fairness audit on Top-10 recommendations. Higher Coverage and Long-tail Share are fairer; lower Gini is fairer.

Model	Coverage@10 ↑	Gini ↓	Long-tail Share@10 ↑
GlobalMean	0.1033	0.9865	0.8727
BiasOnly	0.0946	0.9953	0.2597
ItemKNN	0.8214	0.6899	0.5568
UserKNN	0.7573	0.7489	0.8697
MF	0.0515	0.9937	0.0149
NeuMF	0.4798	0.8762	0.2573
BPR-MF	0.2037	0.9745	0.0047
LightGCN	0.0217	0.9946	0.0000

items and covers 76% of the catalogue, but its ranking accuracy is much lower.

Note that GlobalMean predicts the same score for all items, so its "top-10" is effectively a tie-broken arbitrary subset. Its high Long-tail Share is an artefact of uniform tie-breaking, not a useful recommendation property, and should not be read as a positive result.

The takeaway is that on ML-1M, the accuracy-optimal models concentrate recommendations heavily on popular items, and that optimizing only for NDCG/Precision hides the cost paid by long-tail items. This is a classic accuracy and fairness trade-off and motivates post-hoc re-ranking or fairness-aware training objectives, which we leave as future work.

7 Web Demo

The file `demo/app.py` is a Flask single-page web application that lets the grader interact with any of the seven trained models. The user picks a user ID from the MovieLens-1M universe and selects a model from a dropdown; the server returns the Top-10 recommendation list alongside that user's highest-rated training items (for qualitative inspection) and a static table of test-set metrics for all seven models. Inference is a few milliseconds per user; all pickled models are loaded once at startup. The demo can be launched with `python -m demo.app` and defaults to `http://localhost:5000`. Figure 1 shows the UI running on user 1 with the BPR-MF model selected.

8 Reproducibility

All code is included with this report. The dataset, random seed, hyperparameters, train/test split, and evaluation protocol are all fixed and deterministic. The full pipeline is reproduced by:

```
bash run_all.sh     # train + eval
python -m demo.app  # launch demo
```

The script `run_all.sh` writes metrics and fairness CSVs to `results/`, saves trained checkpoints under `models/`, and recompiles this report and the slides. The repository README contains a file-by-file description of every module.

No external recommender libraries. Per the project policy, we do not import any packaged recommender-system algorithm. The only third-party dependencies are numpy, pandas, scipy, scikit-learn, PyTorch (using only `nn.Embedding`, `nn.Linear`, and `optim.Adam` as building blocks), tqdm, matplotlib, and flask. Every similarity formula, every training loop, every evaluation metric, and every fairness statistic is implemented in this codebase. No call to Surprise, RecBole, LightFM, implicit, or any similar library exists anywhere in the project.

9 Limitations and Future Work

Cold-start serving is not implemented: every model is transductive. Timestamps are unused. Every model is trained with a single seed and no bootstrap confidence intervals are reported. LightGCN's layer count is fixed at $L = 3$; sweeping $L \in \{1, 2, 3, 4\}$ is the most likely fix for the gap to BPR-MF.

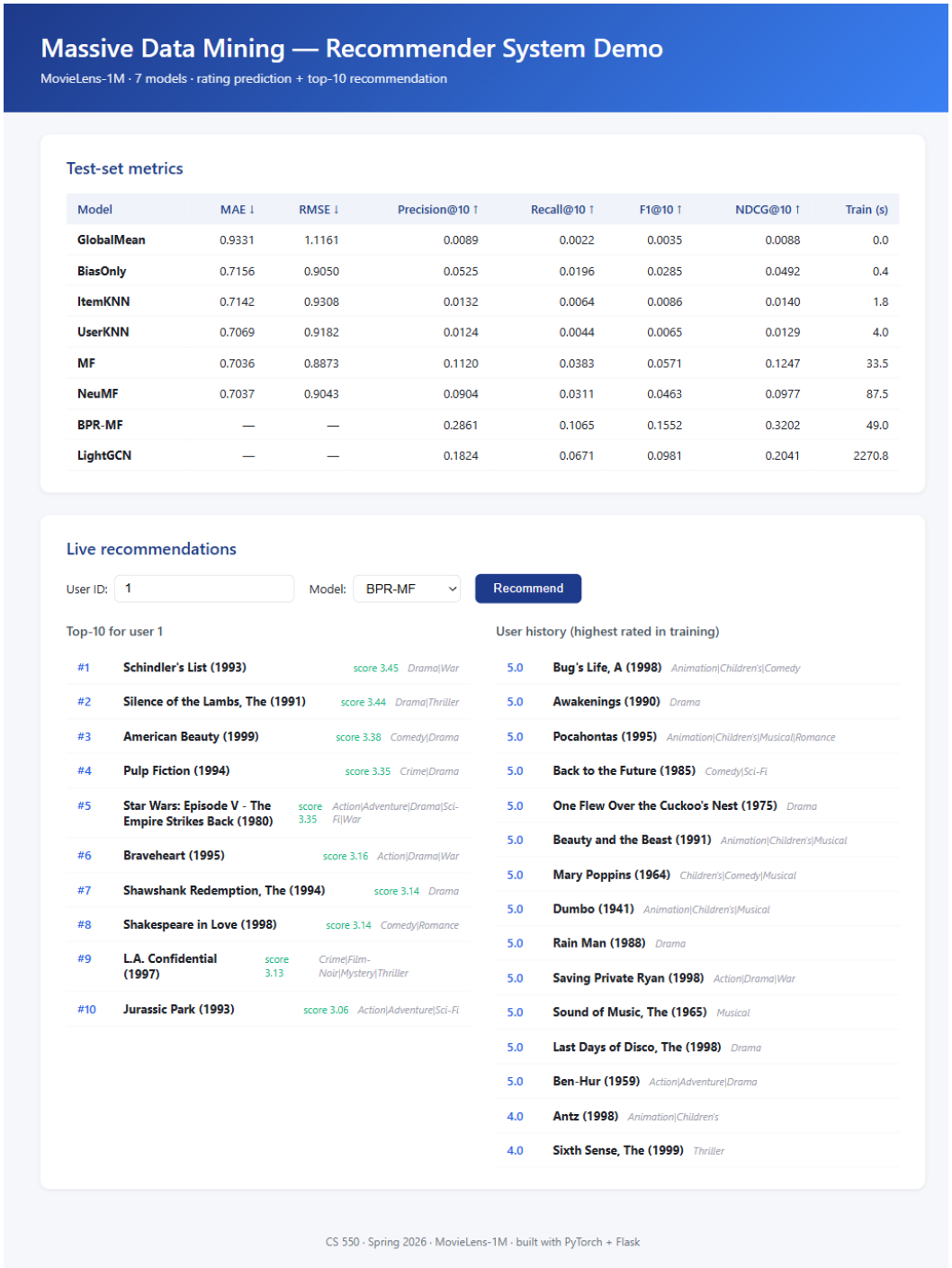

Figure 1: Web demo (`demo/app.py`): test-set metrics for all seven models (top), live Top-10 recommendation for the selected user and model (bottom left), and the user's highest-rated training items for qualitative comparison (bottom right).